专升本 **Java** 程序设计模拟卷-卷 **7**

fifine-专升本

2026 年 3 月 2 日

fifine专升本

一、单项选择题深度解析（共 20 小题）

1. 下列选项中，哪个不是 **Java** 的关键字？

a) class　　b) interface　　c) extends　　d) virtual

答案与深度解析 答案：**D. virtual**

【核心认知框架】

- 题目本质：考查 Java 语言规范（JLS）中保留关键字集合的精确记忆
- 关键区分点：Java 刻意移除了 C++ 中的 `virtual` 关键字，所有非 static 方法默认虚方法
- 命题人意图：测试考生是否混淆 Java 与 C++ 的多态实现机制

【选项原理深度拆解】

- **A. class** —错误（是关键字）
  - 字节码铁证：

```java
// 源代码
public class Test {}
// javap -v 输出
// public class Test extends java.lang.Object
```

  - **JVM** 规范依据：JLS §3.9 明确列出 `class` 为关键字，用于类声明
  - 内存影响：触发类加载器创建 `Class` 对象，分配方法区空间
- **B. interface** —错误（是关键字）
  - 字节码铁证：

```java
// 源代码
public interface Info {}
// javap -v 输出
// public interface Info
// ACC_INTERFACE | ACC\_ABSTRACT
```

  - **JVM** 规范依据：JLS §9.1，接口类型声明必须使用此关键字
  - 设计哲学：Java 单继承多实现的核心载体
- **C. extends** —错误（是关键字）
  - 字节码铁证：

```java
// 源代码
class Sub extends Super {}
// javap 输出
// class Sub extends Super
```

fifine专升本

- **JVM** 规范依据：JLS §8.1.5，类继承声明必需
- 运行时行为：建立父子类关系，影响方法查找表（vtable）构建

• **D. virtual** —正确（非关键字）

- 核心误区澄清：C++ 需显式声明 `virtual`，Java 默认所有实例方法为虚方法
- 字节码证据：

```
// Java 中无需 virtual
public void show() {}
// 字节码标志
// ACC\_PUBLIC (无 ACC\_FINAL 即视为虚方法)
```

- **JVM** 机制：invokevirtual 指令自动处理动态绑定，无需关键字修饰
- 命题陷阱：利用 C++ 背景考生的思维定势

【应背诵知识点（命题人思维版）】

(a) **Java** 保留字清单：共 50+ 个，不含 `virtual`, `goto` (保留但未使用), `const` (保留但未使用)

(b) 多态实现差异：Java 默认虚方法 vs C++ 显式虚函数

【命题趋势与实战策略】

• 考场秒杀：见到 `virtual` 直接选，Java 里没有

• 高频陷阱：`goto` 和 `const` 是保留字但不是有效关键字

2. 以下关于构造方法的描述，正确的是？

a) 构造方法的返回类型必须是 void　b) 构造方法名必须与类名相同　c) 构造方法不能被重载　d) 构造方法不能被自动调用

答案与深度解析 答案：**B.** 构造方法名必须与类名相同

【核心认知框架】

• 题目本质：考查构造方法（Constructor）的语法定义与生命周期行为

• 关键区分点：构造方法无返回值（连 void 都没有），与普通方法的本质区别

• 命题人意图：混淆构造方法与普通方法，测试语法严谨性

【选项原理深度拆解】

• **A.** 构造方法的返回类型必须是 **void** —错误

- 语法硬约束：JLS §8.8 规定构造方法声明不能有返回类型
- 反例验证：

```
public void MyClass() {} // 这是普通方法，不是构造方法！
```

fifine专升本

  - 字节码差异：构造方法对应 `<init>` 方法，普通方法有具体名称
- **B.** 构造方法名必须与类名相同—正确
  - **JVM** 规范依据：JLS §8.8.1，构造方法声明符必须匹配类名
  - 编译期检查：编译器将构造方法转换为 `<init>` 字节码方法
  - 内部类特例：内部类构造方法名包含外部类名（如 `Outer$Inner`）
- **C.** 构造方法不能被重载—错误
  - 核心机制：构造方法支持重载（Overloading），只要参数列表不同
  - 字节码证据：

```
// javap 输出
public <init>(());
public <init>(int); // 重载构造方法
```

  - 应用场景：提供多种对象初始化方式
- **D.** 构造方法不能被自动调用—错误
  - 隐式调用：子类构造方法第一行默认调用 `super()`
  - 字节码证据：

```
// 子类构造方法字节码
0: aload_0
1: invokespecial #1 // 调用父类<init>
```

  - 规则：若未显式调用，编译器自动插入 `super()`

【应背诵知识点（命题人思维版）】

(a) 构造方法特征：无返回值、名同类名、支持重载、对象创建时自动执行

(b) 父子构造顺序：父类构造先于子类构造执行

【命题趋势与实战策略】

- 考场秒杀：看到“返回类型 void”直接排除，构造方法连 void 都不能写
- 高频陷阱：加了 void 就变成了普通方法，不会在 new 时调用

fifine专升本

3. 在 **Java** 中，使用哪个关键字可以调用父类的构造方法？

a) this　　b) super　　c) parent　　d) base

答案与深度解析 答案：**B. super** 【核心认知框架】：考查继承体系中的构造链调用机制。【选项原理深度拆解】：

- **A. this**：调用本类其他构造方法，非父类。
- **B. super**：正确。字节码对应 `invokespecial` 指向父类 `<init>`。
- **C. parent**：Java 无此关键字，常见于其他语言。

- **D. base**：C# 关键字，Java 中无效。

【应背诵知识点】：`super()` 必须是子类构造方法的第一条语句。

4. 下列哪个选项是方法重载的正确示例？

a) `int add(int a, int b)` 和 `double add(int a, int b)`

b) `int add(int a, int b)` 和 `int add(int x, int y)`

c) `int add(int a, int b)` 和 `int add(int a, int b, int c)`

d) `int add(int a, int b)` 和 `void add(int a, int b)`

答案与深度解析 答案：**C** 【核心认知框架】：考查重载（Overloading）的编译期分辨规则。【选项原理深度拆解】：

- **A.** 返回类型不同：错误。重载不看返回类型，会导致编译错误（重复方法）。
- **B.** 参数名不同：错误。参数类型和顺序相同，视为同一方法。
- **C.** 参数个数不同：正确。方法签名（Name + Parameter Types）不同。
- **D.** 返回类型不同：错误。同 A，返回类型不参与签名计算。

【**JVM** 规范依据】：JLS §8.4.2，方法签名仅包含名称和参数类型。

5. 关于继承，以下说法正确的是？

a) Java 支持多重继承

b) 子类可以继承父类的所有成员（包括私有成员）

c) 子类可以访问父类的私有成员

d) 子类可以隐藏父类的静态方法

答案与深度解析 答案：**D** 【核心认知框架】：考查继承的可见性与静态方法绑定机制。【选项原理深度拆解】：

- **A.** 多重继承：错误。Java 类只支持单继承，接口支持多实现。
- **B.** 继承所有成员：错误。私有成员在子类中不可见，虽存在但不可直接访问。
- **C.** 访问私有成员：错误。违反封装性，需通过 getter/setter。
- **D.** 隐藏静态方法：正确。静态方法属于类，子类定义同名静态方法称为隐藏（Hiding），非重写。

【字节码证据】：静态方法调用使用 `invokestatic`，编译期绑定，无多态。

6. 下列哪个访问修饰符修饰的成员只能在同一包中访问？

a) public

b) private

c) protected

d) 默认（无修饰符）

fine专升本

答案与深度解析 答案：**D**【核心认知框架】：考查访问控制权限的边界。【选项原理深度拆解】：

- **A. public**：全局可见。
- **B. private**：仅本类可见。
- **C. protected**：本包 + 子类（即使不同包）。
- **D.** 默认：仅本包可见（Package-Private）。

【**JVM** 规范依据】：JVMS §5.4.4 访问控制检查逻辑。

7. 关于抽象类，以下描述错误的是？

a) 抽象类可以包含抽象方法　b) 抽象类不能实例化　c) 抽象类可以包含构造方法　d) 抽象类中的抽象方法必须在子类中实现

答案与深度解析 答案：**D**【核心认知框架】：考查抽象类的约束条件与例外情况。【选项原理深度拆解】：

- **A.** 含抽象方法：正确。
- **B.** 不能实例化：正确。字节码中 `ACC_ABSTRACT` 标志禁止 `new`。
- **C.** 含构造方法：正确。供子类调用初始化。
- **D.** 必须在子类实现：错误。若子类也是抽象类，则无需实现。

【命题陷阱】：忽略“子类也是抽象类”的递归情况。

fitme专升本

8. 接口中的方法默认的修饰符是？

a) public abstract　b) public　c) abstract　d) private

答案与深度解析 答案：**A**【核心认知框架】：考查接口成员的隐式修饰符。【选项原理深度拆解】：

- **A. public abstract**：正确。JLS §9.4 规定接口方法默认为公共抽象。
- **B. public**：不完整，缺少 abstract。
- **C. abstract**：不完整，缺少 public。
- **D. private**：JDK9 支持 private 方法，但非默认。

【字节码证据】：编译后自动添加 `ACC_PUBLIC | ACC_ABSTRACT`。

9. 下列关于 `final` 关键字的说法，正确的是？

a) final 修饰的类可以被继承
b) final 修饰的方法可以被重写
c) final 修饰的变量必须在声明时赋值,之后不能修改
d) final 修饰的引用变量可以改变指向的对象

答案与深度解析 答案：**C** 【核心认知框架】：考查 final 的不可变性语义。【选项原理深度拆解】:

- **A.** 类可继承：错误。final 类禁止继承（如 String）。
- **B.** 方法可重写：错误。final 方法禁止重写，保障行为不变。
-
- **C.** 变量不可修改：正确。一次赋值原则（Definite Assignment）。
- **D.** 引用可变：错误。final 引用指向不可变，但对象内容可变。

【**JMM** 保障】：final 域写入后建立冻结屏障，保证可见性。

10. 下列哪个类是所有 **Java** 类的根类？

a) Object
b) Class
c) System
d) Main

答案与深度解析 答案：**A** 【核心认知框架】：考查类继承体系的顶层结构。【选项原理深度拆解】:

- **A. Object**：正确。JLS §4.3.2，所有类隐式继承 Object。
- **B. Class**：类的元数据描述类。
- **C. System**：系统工具类。
- **D. Main**：非标准类。

【字节码证据】：任何类 javap 输出均显示 `extends java/lang/Object`。

11. 以下关于 `String` 类的描述，错误的是？

a) String 对象是不可变的
b) 使用 == 比较两个字符串时,比较的是引用是否相等
c) String 类位于 java.lang 包中
d) String 类可以被继承

答案与深度解析 答案：**D** 【核心认知框架】：考查 String 类的不可变性与 final 修饰。【选项原理深度拆解】:

- **A.** 不可变：正确。内部 char[] 或 byte[] 为 final。
- **B.** == 比较引用：正确。对象比较默认地址。
- **C. java.lang**：正确。核心类库。

ffine专升本

- **D.** 可继承：错误。String 是 final 类，禁止继承保障安全。

【安全考量】：防止子类篡改字符串内容，保障字符串常量池安全。

12. 下列哪个异常是数组下标越界时抛出的？

a) NullPointerException b) ArrayIndexOutOfBoundsException c) IndexOutOfBoundsException d) IllegalArgumentException

答案与深度解析 答案：**B** 【核心认知框架】：考查运行时异常的具体类型。【选项原理深度拆解】：

- **A. NPE**：空指针。
- **B. AIOOBE**：正确。数组特有。
- **C. IOOBE**：父类，List 等也抛此异常，但数组具体为 B。
- **D. IAE**：参数非法。

【**JVM** 机制】：数组访问指令 `aaload` 等会自动检查边界并抛出。

13. 在 **try-catch-finally** 结构中，关于 **finally** 块的执行，正确的是？

a) 只有在 try 块没有异常时才会执行

b) 只有在 catch 块执行后才会执行

c) 无论是否发生异常都会执行（除非 JVM 退出）

d) 可以没有 try 块单独存在

答案与深度解析 答案：**C** 【核心认知框架】：考查异常处理流程控制。【选项原理深度拆解】：

- **A/B.** 条件执行：错误。finally 设计初衷即为清理资源，必执行。
- **C.** 总是执行：正确。例外：`System.exit(0)` 或断电。
- **D.** 单独存在：错误。语法错误，必须配合 try。

【字节码证据】：Exception Table 中 finally 块对应特殊的跳转逻辑。

14. 下列哪个集合类实现了动态数组，且允许存储重复元素？

a) HashSet b) TreeSet c) ArrayList d) HashMap

答案与深度解析 答案：**C** 【核心认知框架】：考查集合框架的实现结构。【选项原理深度拆解】：

- **A/B. Set**：不允许重复。
- **C. ArrayList**：正确。基于数组，允许重复，有序。
- **D. Map**：键值对，非单列集合。

【内存结构】：ArrayList 内部维护 `Object[] elementData`。

fifine专升本

15. 关于泛型，以下说法错误的是？

a) 泛型可以提高类型安全
b) 泛型可以用于类、接口和方法
c) 泛型类型参数在运行时会被类型擦除
d) 泛型可以用于基本数据类型

答案与深度解析 答案：**D** 【核心认知框架】：考查泛型的类型擦除机制与限制。【选项原理深度拆解】:

- **A.** 类型安全：正确。编译期检查。
- **B.** 适用范围：正确。
- **C.** 类型擦除：正确。运行时变为 Object 或边界类型。
- **D.** 基本类型：错误。必须使用包装类（如 `Integer`）。

【**JVM** 限制】: 泛型实现基于对象，基本类型非对象。

16. 在 **Java** 中，创建线程的方式不包括？

a) 继承 Thread 类
b) 实现 Runnable 接口
c) 实现 Callable 接口
d) 继承 Process 类

答案与深度解析 答案：**D** 【核心认知框架】: 考查多线程实现 API。【选项原理深度拆解】:

- **A/B/C**：均为合法创建方式（Callable 需配合 FutureTask）。
- **D. Process**：错误。Process 用于管理操作系统进程，非线程。

【进程 **vs** 线程】: Process 重量级，Thread 轻量级。

17. 关于 **synchronized** 关键字的说法，正确的是？

a) 可以修饰类
b) 可以修饰构造方法
c) 可以修饰静态方法
d) 可以修饰成员变量

答案与深度解析 答案：**C** 【核心认知框架】: 考查同步锁的修饰范围。【选项原理深度拆解】:

- **A.** 修饰类：错误。语法不支持。
- **B.** 修饰构造：错误。构造方法线程安全无意义（对象未共享）。
- **C.** 修饰静态方法：正确。锁住 Class 对象。
- **D.** 修饰变量：错误。需修饰代码块或方法。

【字节码证据】: `ACC_SYNCHRONIZED` 标志位。

18. 下列哪个类用于读取文本文件中的字符数据？

fifine专升本

a) FileInputStream b) FileReader c) FileWriter d) BufferedInputStream

答案与深度解析 答案：**B**【核心认知框架】：考查 IO 流字节与字符的区别。【选项原理深度拆解】：

- **A/D.** 字节流：适合二进制，读文本需转换。
- **B. FileReader**：正确。字符输入流。
- **C. FileWriter**：输出流。

【编码处理】：FileReader 使用默认编码，推荐 InputStreamReader 指定编码。

19. 在 **JDBC** 中，用于执行 **SQL** 语句并返回结果集的对象是？

a) Statement b) PreparedStatementc) ResultSet d) Connection

答案与深度解析 答案：**A** 或 **B** (通常指 **Statement** 接口) 注：题目略有歧义，Statement 和 PreparedStatement 均可，但 ResultSet 是结果。根据常规题库，选 Statement 代表执行者。若单选且强调返回结果集的对象，通常指执行者。此处修正为考查执行者。修正解析：题目问“执行... 并返回”，执行者是 Statement 系列，返回的是 ResultSet。若问返回结果集的对象类型，选 ResultSet。若问执行者，选 Statement。根据选项分布，通常考查 Statement。答案：**A** (代表执行接口)【核心认知框架】：考查 JDBC 核心接口职责。【选项原理深度拆解】：

- **A. Statement**：正确。执行静态 SQL。
- **B. PreparedStatement**：也是执行者，预编译。
- **C. ResultSet**：结果集数据，非执行者。
- **D. Connection**：连接对象。

fifine专升本

20. 关于 **Swing** 中 **JFrame** 的默认布局管理器是？

a) FlowLayout b) BorderLayout c) GridLayout d) CardLayout

答案与深度解析 答案：**B**【核心认知框架】：考查 Swing 组件默认布局。【选项原理深度拆解】：

- **A. FlowLayout**：Panel 默认。
- **B. BorderLayout**：正确。JFrame 内容面板默认。
- **C/D.** 其他：需手动设置。

二、判断题深度解析（共 10 小题）

1. 一个 **Java** 源文件中可以定义多个 **public** 类。 ___(B)

深度解析 答案：错误 【**JVM** 规范依据】：JLS §7.6 规定每个编译单元（源文件）只能有一个 public 类，且文件名必须与该类名相同。【字节码证据】：编译器生成.class 文件时，public 类名必须匹配文件名，否则报错。【例外情况】：非 public 类可以有多个。

2. 静态方法可以直接调用非静态成员变量。 ___(B)

深度解析 答案：错误 【内存模型】：静态方法属于类（方法区），非静态变量属于对象（堆）。静态方法执行时可能无对象实例。【编译错误】：`Cannot make a static reference to the non-static field`。【解决方案】：需通过对象实例引用。

3. 子类构造方法中，如果没有显式调用父类构造方法，则默认调用父类的无参构造方法。 ___(A)

深度解析 答案：正确 【字节码证据】：编译器自动插入 `invokespecial` 调用父类 `<init>`。【陷阱】：若父类无无参构造，编译报错，需显式调用 `super(args)`。

4. 接口中的成员变量默认是 **public static final** 的。 ___(A)

深度解析 答案：正确 【**JLS** 依据】：JLS §9.3 字段声明。【字节码】：编译后自动添加 `ACC_PUBLIC | ACC_STATIC | ACC_FINAL`。

5. 使用 `throw` 关键字抛出异常时，方法声明中必须使用 `throws`。 ___(B)

深度解析 答案：错误 【异常分类】：仅受检异常（Checked Exception）必须声明。运行时异常（RuntimeException）无需声明。【**JVM** 机制】：运行时异常不强制捕获或声明。

6. **HashSet** 中的元素是有序的（按插入顺序）。 ___(B)

深度解析 答案：错误 【数据结构】：HashSet 基于 HashMap，无序。【对比】：LinkedHashSet 才维护插入顺序。

7. 泛型通配符 `?` 可以用于方法参数，表示接受任意类型的泛型集合。 ___(A)

tfine专升本

深度解析 答案：正确 【用法】：`List<?>` 表示未知类型的列表。【限制】：只能读不能写（除 null）。

8. 线程调用 **sleep()** 方法后不会释放持有的对象锁。 ___(A)

深度解析 答案：正确 【机制对比】：`sleep()` 不释放锁，`wait()` 释放锁。【应用场景】：需暂停但不释放资源时用 sleep。

9. **File** 类的 **delete()** 方法可以删除非空目录。 ___(B)

深度解析 答案：错误 【**API** 行为】：delete() 只能删除空目录或文件。【解决方案】：需递归删除目录下所有文件。

10. 在 **UDP** 通信中，**DatagramSocket** 用于发送和接收数据报。___(A)

深度解析 答案：正确 【网络编程】：UDP 无连接，使用 DatagramSocket 包裹 DatagramPacket。

fime专升本

三、填空题深度解析（共 5 小题）

1. Java 语言中，定义包使用关键字 ____，导入包使用关键字 ____。

深度解析 答案：**package, import** 【字节码影响】：`package` 决定类的全限定名，影响类加载器查找路径。`import` 仅编译期辅助，不进入字节码。【规范】：package 必须是源文件第一行（注释除外）。

2. 面向对象的三大特征是封装、____ 和 ____。

深度解析 答案：继承，多态 【核心认知】：封装（隐藏细节）、继承（代码复用）、多态（接口多样实现）。【**JVM** 体现】：多态通过 vtable 和 invokevirtual 指令实现。

3. 异常分为两大类：____ 异常和 ____ 异常（运行时异常）。

深度解析 答案：受检（**Checked**），非受检（**Unchecked**）【处理机制】：受检异常强制处理，非受检异常可选。【继承体系】：Exception vs RuntimeException。

4. 线程的生命周期包括新建、____、运行、____ 和死亡五种状态。

深度解析 答案：就绪，阻塞 【状态图】：New -> Runnable -> Running -> Blocked/Waiting -> Terminated。【**OS** 映射】：Java 线程状态映射到操作系统线程状态。

5. Java 集合框架中，List 接口的实现类有 ____ 和 ____。

深度解析 答案：**ArrayList, LinkedList** (或 Vector/Stack)【结构差异】：ArrayList (数组，随机访问快)，LinkedList (链表，插入删除快)。【选型策略】：读多用 ArrayList，写多用 LinkedList。

四、程序运行结果题深度解析（共 5 小题）

1. 写出下列程序的输出结果：

```
public class Test1 {
public static void main(String[] args) {
int x = 10, y = 20;
if (x++ > 10 || ++y > 20) {
System.out.print(x + "," + y);
} else {
System.out.print(x + "," + y);
}
}
}
```

深度解析 输出：**11,20** 【执行流程分析】：

- 短路逻辑：`||` 左侧 `x++ > 10`。先取值 10，比较 10>10 为 false，x 变为 11。
- 右侧执行：左侧 false，执行右侧 `++y > 20`。y 先加 1 变 21，比较 21>20 为 true。
- 结果：条件整体 true，进入 if 块。输出 x=11, y=21。
- 修正：等等，++y 后 y 为 21。输出应为 11,21。
- 再次检查：x++ 后 x=11。条件 true。输出 11,21。
- 注意：若题目意图考查短路，若左侧为 true，右侧不执行。此处左侧 false，右侧执行。

【正确答案修正】：11,21 【字节码视角】：`ifeq` 指令实现短路跳转。

2. 写出下列程序的输出结果：

fifine专升本

```java
class A { public void show() { System.out.print("A"); } }
class B extends A { public void show() { System.out.print("B"); } }
public class Test2 {
public static void main(String[] args) {
A obj = new B();
obj.show();
}
}
```

深度解析 输出：**B** 【多态机制】：编译时类型 A，运行时类型 B。【字节码证据】：invokevirtual 指令根据对象实际类型查找 vtable。【核心考点】：动态绑定（Dynamic Binding）。

3. 写出下列程序的输出结果：

```java
public class Test3 {
public static void main(String[] args) {
try {
int[] a = new int[5];
a[5] = 10;
System.out.println("正常");
} catch (ArrayIndexOutOfBoundsException e) {
System.out.println("数组越界");
} catch (Exception e) {
System.out.println("其他异常");
} finally {
System.out.println("结束");
}
}
}
```

深度解析 输出：数组越界 结束 【异常匹配】：a[5] 抛出 ArrayIndexOutOfBoundsException。【**catch** 顺序】：子类异常在前，匹配成功。【**finally** 保障】：无论异常与否，finally 必执行。

4. 写出下列程序的输出结果：

```java
public class Test4 {
public static void main(String[] args) {
```

tifine专升本

```
String s1 = "hello";
String s2 = new String("hello");
System.out.println(s1 == s2);
System.out.println(s1.equals(s2));
}
}
```

深度解析 输出：false true【内存模型】：s1 在常量池，s2 在堆。地址不同，== 为 false。【内容比较】：equals 比较字符内容，为 true。【字符串池】：new String 强制创建新对象。

5. 写出下列程序的输出结果：

```
class Counter {
static int count = 0;
Counter() { count++; }
}
public class Test5 {
public static void main(String[] args) {
new Counter();
new Counter();
Counter c = new Counter();
System.out.println(Counter.count);
}
}
```

fifine专升本

深度解析 输出：3【静态变量】：count 属于类，所有实例共享。【执行计数】：new 三次，构造方法执行三次，count 累加至 3。【内存区域】：count 存储在方法区（静态区）。

五、编程题深度解析与设计规范（共 2 小题）

1. 设计一个" 人" 类及其子类

深度解析与标准实现 【设计模式】：模板方法模式的变体，利用抽象类定义骨架。【核心考点】：抽象类、继承、多态、封装。【标准代码】：

```
abstract class Person {
    private String name;
```

```java
    private int age;
    // 构造、Getter/Setter 省略...
    public abstract void work();
}
class Student extends Person {
    // 实现 work()
}
// 多态测试
Person[] people = new Person[]{new Student(), new Teacher()};
for(Person p : people) p.work(); // 动态绑定
```

【最佳实践】:

- 成员变量私有化，保障封装。
- 抽象方法强制子类实现，保障行为规范。
- 使用父类引用接收子类对象，体现多态。

2. 实现一个简单的银行账户类

深度解析与标准实现 【设计原则】: 原子性、一致性（余额不为负）。【线程安全】: 若多线程访问，deposit/withdraw 需 synchronized。【标准代码】:

```java
public class BankAccount {
    private String accountNumber;
    private double balance;
    // 构造方法逻辑...
    public synchronized void deposit(double amount) {
        if(amount > 0) balance += amount;
    }
    public synchronized void withdraw(double amount) {
        if(amount > 0 && amount <= balance) balance -= amount;
    }
}
```

【关键逻辑】:

- 金额校验：防止负数操作。
- 余额校验：防止透支。
- 同步锁：保障并发安全（题目虽未明示，但为高分点）。

——讲义结束，祝学习顺利——

fifine专升本